

Основы языка описания аппаратуры VHDL.

1. Введение.

Языки описания аппаратуры (Hardware Description Language – HDL) используются для формального представления структуры и алгоритма работы цифровых электронных систем и их составных частей. Подобные описания систем используются, как правило, для компьютерного моделирования их функционирования, а также для синтеза описываемых систем средствами программируемых логических интегральных схем (ПЛИС). Основными языками описания аппаратуры, используемыми в настоящее время, являются такие языки, как VHDL и System Verilog. В данном курсе дается краткое введение в язык VHDL, в объеме, достаточном для моделирования цифровых систем, изучаемых в базовых курсах по изучению цифровой и вычислительной техники.

2. Структура описания системы.

Базовым элементом описания системы на VHDL является т.н. «объект» (Entity). Под этим термином подразумевается некое конкретное устройство, подлежащее описанию. В качестве такого устройства может подразумеваться простейший логический вентиль, комбинационное или последовательностное логическое устройство, блок электронной памяти, контроллер, процессор и т.д. Каждый объект обладает уникальным именем, а также набором внешних сигналов, с помощью которых он взаимодействует с другими внешними по отношению к нему системами. Пример задания объекта представлен ниже.

```
Entity andnot is
Port (a:in std_logic;
      b:in std_logic;
      q:out std_logic;
End andnot;
```

```
Entity andnot_n is
Generic (Data_Width: integer:= 8);
Port (a:in std_logic_vector(Data_Width-1 downto 0);
      b:in std_logic_vector(Data_Width-1 downto 0);
      q:out std_logic_vector(Data_Width-1 downto 0);
End andnot_n;
```

Директива **Entity** задает новый объект описания. В данном случае объект имеет имя «andnot».

Имя объекта, как и другие идентификаторы, используемые при описании, должно начинаться с латинской буквы и может содержать только латинские буквы, цифры и символ подчеркивания. Регистр символа, используемого в идентификаторе, значения не имеет. Это значит, что идентификаторы andnot и AnDNoT идентичны.

Необязательная директива **Generic** дает возможность задавать параметры объекта, позволяющие масштабировать или модифицировать объект без необходимости изменять весь код описания объекта.

Директива **Port** задает перечень внешних сигналов объекта. Для каждого сигнала задается его имя (a, b, q), направленность сигнала (**in** – входной сигнал, **out** – выходной сигнал, **inout** – двунаправленный сигнал), тип сигнала.

Алгоритм функционирования объекта или его структура описывается в блоке **Architecture**. Пример данного блока представлен ниже.

Architecture behave of andnot is

Begin

q <= '0' when ((a = '1') and (b='1')) else '1';

End Architecture behave;

Для каждого объекта можно задать несколько различных блоков **Architecture**.

Architecture behave1 of andnot is

Begin

q <= not (a and b);

End Architecture behave1;

Каждый блок **Architecture** должен иметь уникальное имя (**behave**, **behave1**) и должен заканчиваться директивой **End Architecture**. Внутри блока **Architecture** описание объекта начинается с директивы **Begin**. До нее могут объявляться сигналы, переменные и константы, используемые внутри объекта.

3. Сигналы, переменные, константы.

Цифровые значения в VHDL могут быть представлены в виде целых или вещественных чисел в десятичной системе счисления, а также в других системах счисления. Примеры задания цифровых значений представлены ниже.

Целые числа в десятичной системе счисления: 12, 127, 14E3 ($14 \cdot 10^3$), 45_000 (45000).

Вещественные числа в десятичной системе счисления: 12.5, 0.623, 32.7E-5.

Числа в других системах счисления: 2#1001_0101# (149), 16##f# (255), b"1001" (9), 3b"101" (5), 8x"aa" ($2 \cdot 10101010 = 170$).

Для определения константы используется директива **constant**. Примеры определения констант приведены ниже.

constant offset: real:= 0.5;

constant setup_value: std_logic:= '0';

constant Data_Init_Value: std_logic_vector(7 downto 0):= ("00000000");

constant Data_Init_Value1: std_logic_vector(7 downto 0):= ('1', '1', others => '0');

В языке VHDL различают переменные и сигналы. Значения переменных аналогично различным языкам программирования изменяются соответствующими операторами в момент их выполнения. Переменные могут быть определены в области операторов процессов, функций и процедур. Переменные задаются с помощью директивы **variable**. Пример задания переменной приведен ниже.

Variable count: natural:=0;

Для каждой переменной задается имя (count), тип (natural), а также может задаваться начальное значение.

Сигналы описывают реальные информационные связи между составными частями объекта и между объектом и другими объектами. Сигналу можно поставить в соответствие реальный провод, соединяющий определенный выход объекта (или его части) с определенными входами. Сигналы не могут быть определены внутри процессов, функций и процедур. Они определяются только внутри тела описания архитектуры объекта. Изменение сигналов ассоциировано с конкретными моментами времени.

Сигналы отличаются от переменных тем, что, как в реальной системе могут изменяться и обрабатываться параллельно. Внешние сигналы объекта задаются при его описании директивой **Entity**. Внутренние сигналы объекта задаются директивой **signal**. Пример задания сигнала представлен ниже.

Signal c, d: std_logic;

Для каждого сигнала задается имя (c, d), тип (std_logic).

Базовыми типами переменных и сигналов являются:

- **integer** – целые значения;
- **real** – вещественные значения;
- **character** – символьные значения;
- **string** – строковые значения.

Директивы **type** и **subtype** позволяют задавать собственные типы данных. Примеры производных типов приведены ниже.

Тип **bit** может принимать одно из двух символьных значений '0' или '1'.

type bit is ('0', '1');

Тип **boolean** может принимать одно из двух значений false или true.

type boolean is (false, true);

Тип **time** задает физический тип времени, используемый при моделировании системы.

type time is range 0 to 1E20

units

fs;

ps = 1000fs;

ns = 1000ps;

us = 1000ns;

ms = 1000us;

s = 1000ms;

end units;

Физические типы могут иметь единицы измерения. Они задаются директивой **units**. В данном случае задаются базовая единица измерения (фс) и производные единицы измерения.

Тип **natural** может принимать значение в диапазоне натуральных чисел.

subtype natural is integer range 0 to highest_integer;

Тип **positive** может принимать значение в диапазоне положительных чисел.

subtype positive is integer range 1 to highest_integer;

Переменные и сигналы могут объединяться в массивы с помощью директивы **array**.

type bit_vector is array (natural <>) of bit;

signal data_in: bit_vector(0 to 15);

signal data_out: bit_vector(7 downto 0);

type two_dimension is array (natural <>, natural <>) of bit;

signal data_matrix: two_dimension(15 downto 0, 7 downto 0);

Большинство обычных цифровых сигналов подразумевают, что каждый из них формируется в одном определенном источнике («драйвере») и может подаваться на несколько различных входов в данном объекте или в других объектах. Такие сигналы принимают одно из двух возможных значений '0' или '1'. Однако, иногда в цифровых системах возникают ситуации, когда несколько различных выходов объединяются вместе. Примером может служить шина данных микропроцессорной системы. Для того, чтобы это стало возможным все объединяемые выходы должны иметь возможность переходить в т.н. «высокоимпедансное» состояние. Такое состояние обозначается как 'z'. Кроме того, при некорректной работе подобных систем на одном из объединенных драйверов может выставиться «единица», а на другом «ноль». При этом уровень данного сигнала является некорректным. Такое состояние обозначается как 'x'. Как известно, выходы элементов электронной памяти, таких, как, например триггер, при включении равновероятно могут принимать значения '0' или '1'. Если не произвести принудительную инициализацию подобных устройств, состояние их выходов является неопределенным. Неинициализированное состояние сигнала обозначается как 'u'. Тип сигнала, который может принимать все описанные значения, называется **std_logic**. Это самый распространенный тип, используемый для описания цифровых сигналов. Сигнал этого типа может принимать одно из следующего списка значений:

- '1' – логическая единица;
- '0' – логический ноль;
- 'Z' – высокоимпедансное состояние;
- 'W' - сигнал с «подтяжкой» (через резистор) к неизвестному уровню;
- 'L' - сигнал с «подтяжкой» (через резистор) к нулю;

- 'H' сигнал с «подтяжкой» (через резистор) к единице;
- '-' – произвольное значение сигнала;
- 'U' - неинициализированное значение сигнала;
- 'X' – неизвестное значение сигнала.

Сигналы типа **std_logic** относятся к т.н. «разрешаемым» сигналам. Это означает, что они имеют т.н. «функцию разрешения». Её смысл заключается в том, что если несколько сигналов такого типа объединены вместе и имеют одновременно разное значение, то функция разрешения определяет значение результирующего сигнала. Функция разрешения **std_logic** для двух источников A и B имеет вид, представленный ниже.

		A									
B		‘U’	‘X’	‘0’	‘1’	‘Z’	‘W’	‘L’	‘H’	‘-’	
	‘U’	‘U’	‘U’	‘U’	‘U’	‘U’	‘U’	‘U’	‘U’	‘U’	
	‘X’	‘U’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	
	‘0’	‘U’	‘X’	‘0’	‘X’	‘0’	‘0’	‘0’	‘0’	‘X’	
	‘1’	‘U’	‘X’	‘X’	‘1’	‘1’	‘1’	‘1’	‘1’	‘X’	
	‘Z’	‘U’	‘X’	‘0’	‘1’	‘Z’	‘W’	‘L’	‘H’	‘X’	
	‘W’	‘U’	‘X’	‘0’	‘1’	‘W’	‘W’	‘W’	‘W’	‘X’	
	‘L’	‘U’	‘X’	‘0’	‘1’	‘L’	‘W’	‘L’	‘W’	‘X’	
	‘H’	‘U’	‘X’	‘0’	‘1’	‘H’	‘W’	‘W’	‘H’	‘X’	
	‘-’	‘U’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	‘X’	

Пример задания сигналов типа **std_logic** приведен ниже.

signal a, b: std_logic;

Сигналы подобного типа могут объединяться в массивы. Для этого используется тип **std_logic_vector**, который задается как показано ниже.

type std_logic_vector is array (natural range <>) of std_logic;

signal Data_Bus: std_logic_vector(15 downto 0);

Типы **std_logic**, **std_logic_vector** и их функция разрешения определены в пакете **std_logic_1164** библиотеки **ieee**. Поэтому перед определением сигналов данных типов обязательно нужно подключить данную библиотеку и пакет, как показано ниже.

library ieee;

use ieee.std_logic_1164.all;

Более подробно об использовании библиотек и пакетов будет рассказано далее.

Сигналам, массивам и другим именованным элементам приписывается ряд т.н. «атрибутов». Атрибут - некое свойство элемента, которое может быть использовано для описания алгоритма работы объекта.

В случае сигналов самым часто используемым является атрибут **'event**. Он принимает значение **true** в те моменты времени, когда сигнал, к которому он относится, меняет свое значение. В остальные моменты времени он принимает значение **false**. Например, для сигнала CLK этот атрибут записывается как **CLK'event**. С сигналами связан также ряд функций. Наиболее часто используемыми из них являются функции **rising_edge** и **falling_edge**. Функция **rising_edge** принимает значение **true** в момент перепада сигнала из состояния '0' в состояние '1', а функция **falling_edge** принимает значение **true** в момент перепада сигнала из состояния '1' в состояние '0'. В остальные моменты эти функции принимают значение **false**.

Часто используемыми атрибутами массивов являются следующие:

- **'left** – левая граница диапазона индекса массива;
- **'right** – правая граница диапазона индекса массива;
- **'low** – нижняя граница диапазона индекса массива;

- '**high**' – верхняя граница диапазона индекса массива;
- '**range(n)**' – диапазон индексов разрядности n;
- '**length(n)**' – длина диапазона индексов разрядности n.

Примеры атрибутов массивов приводятся ниже.

```
signal Data:std_logic_vector(15 downto 0);
```

```
Data'left = Data'high = 15
```

```
Data'right = Data'low = 0
```

```
Data'range(1) = 15 downto 0
```

```
Data'lenght(1) = 16
```

Описание других атрибутов сигналов можно найти в [1] - [5].

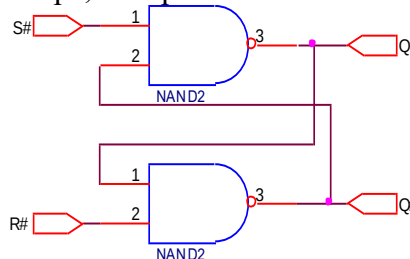
4. Способы описания архитектуры объекта.

Архитектура объекта определяет в том или ином виде выполняемую им функцию. Различают поведенческий и структурный способ описания архитектуры объекта. Проиллюстрируем оба способа на реальных примерах. Используемые в примерах операторы интуитивно понятны и будут описаны позже. В качестве примера поведенческого описания объекта рассмотрим логический элемент «И-НЕ».

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
Entity andnot is  
Port (a:in std_logic;  
      b:in std_logic;  
      q:out std_logic);  
End andnot;  
Architecture behav of andnot is  
Begin  
q <= not (a and b);  
End Architecture behav;
```

В представленном примере объявлен объект andnot, имеющий два входа (a и b) и один выход q. В блоке описания архитектуры после директивы **Begin** представлен т.н. оператор параллельного присваивания сигнала (будет описан позже), который устанавливает связь выхода q со входами a и b с помощью соответствующей логической функции.

В качестве примера структурного способа описания архитектуры объекта, рассмотрим описание RS-триггера, построенного по схеме представленной ниже.



```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
Entity rs_trg is  
Port (r:in std_logic;  
      s:in std_logic;  
      q:out std_logic;  
      qn:out std_logic);  
End rs_trg;  
Architecture behav_rs_trg of rs_trg is  
Component andnot is
```

```

Port (a:in std_logic;
      b:in std_logic;
      q:out std_logic);
End Component andnot;
signal q_int: std_logic;
signal qn_int: std_logic;
Begin
nand1: andnot
Port Map (a=>s,
          b=>qn_int,
          q=>q_int);
nand2: andnot
Port Map (a=>r,
          b=>q_int,
          q=>qn_int);

q <= q_int;
qn <= qn_int;
End Architecture behav_rs_trg;

```

В представленном примере объявлен объект `rs_trg`, который имеет два входа `r` и `s`, а также два выхода `q` и `qn`. В блоке описания архитектуры объявлен компонент, который будет использован для построения объекта `rs_trg`. В качестве этого компонента используется объект `andnot`, описанный в предыдущем примере. Как видно из схемы триггера, он состоит из двух элементов «И-НЕ», входы и выходы которых соединены определенным образом. В соответствии с этим с помощью директивы **Component** объявляется объект `nand`, описанный выше, который будет использован для создания объекта `rs_trg`. Объявление компонента завершается директивой **End Component**. После директивы **Begin** конкретизируются (объявляются) два экземпляра компонента `andnot`, имеющие метки `nand1` и `nand2`. Директивы **Port Map** в каждом случае задают подключения входов и выходов соответствующих экземпляров компонента с сигналами объекта `rs_trg`. Если при объявлении объекта, используемого в качестве компонента, использовалась директива `Generic`, то при объявлении каждого экземпляра компонента должна быть использована директива **Generic Map**, в которой используемым параметрам ставятся в соответствие конкретные значения. Пример такой директивы приведен ниже.

Generic Map(Data_Width => 16)

Помимо компонента `nand` в блоке описания архитектуры объекта заданы два сигнала `q_int` и `qn_int`. Их необходимость обусловлена тем, что сигналы `q` и `qn` объявлены как выходы объекта `rs_trg`, а значит, они не могут быть использованы как входы для экземпляров компонента `nand`. Эта проблема легко решается использованием внутренних сигналов `q_int` и `qn_int`, которые могут быть использованы и как входы и как выходы. Выходы `q` и `qn` соединяются с этими внутренними сигналами с помощью двух операторов параллельного присваивания сигналов в конце блока описания архитектуры объекта `rs_trg`. Как можно заметить, в данном случае архитектура объекта описана не в виде алгоритма его функционирования, а в виде структуры объекта.

Представленное структурное описание RS-триггера вовсе не является безальтернативным. Ниже представлено поведенческое описание RS-триггера.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;
Entity rs_trg1 is
Port (r:in std_logic;
      s:in std_logic;
      q:out std_logic;
      qn:out std_logic);

```

```

End rs_trg1;
Architecture behav of rs_trg1 is
signal q_int: std_logic;
Begin
Process (r, s)
Begin
    if ((r = '0') and (s = '1')) then q_int <= '0';
    else    if ((s = '0') and (r = '1')) then q_int <= '1';
            end if;
    end if;
End process;
q <= q_int;
qn <= not q_int;
End Architecture behav;

```

В данном примере объявлен объект, аналогичный предыдущему. В блоке описания архитектуры с помощью директивы **Process** объявляется т.н. «процесс», который описывает некоторую независимое поведение некоторой части объекта, направленное на формирование определенного (или определенных) сигналов объекта. В директиве **Process** в скобках приводится т.н. лист ожидания процесса. В нем перечисляются сигналы, изменение которых запускает данный процесс. Если сигналы, перечисленные в списке ожидания, не меняют своего значения, то процесс находится в пассивном состоянии. Если какой-либо из сигналов, перечисленных в списке ожидания меняет свое значение, то процесс активизируется и выполняются операторы, указанные в нем. В данном случае в теле процесса представлены интуитивно понятные операторы **if**, которые меняют значение сигнала **q_int** в зависимости от комбинации значений входных сигналов. Инверсный выход **qn** формируется как логическая инверсия **q_int** с помощью оператора **not**.

Справедливости ради нужно отметить, что два представленных описания RS-триггера не являются полностью эквивалентными в смысле функционирования объекта. Для первого структурного описания комбинация входных сигналов **r=0, s=0** является «запрещенной», как у классического RS-триггера, и приводит к нестабильному состоянию устройства. В тоже время, второе поведенческое описание RS-триггера допускает эту комбинацию входных сигналов. При этом сохранится предыдущее состояние выходов.

5. Операции.

Для реализации вычислений в VHDL предусмотрено большое число арифметических и логических операций. Большинство из них сведены в следующей таблице.

Операция	Запись	Пример	Результат
Сложение	$A + B$	$2 + 3$	5
Вычитание	$A - B$	$5 - 2$	3
Умножение	$A * B$	$5 * 3$	15
Деление нацело	A / B	$5 / 3$	1
Модуль	$A \bmod B$	$10 \bmod 4$	2
		$10 \bmod (-4)$	-2
Остаток	$A \text{ rem } B$	$10 \text{ rem } 4$	2
		$10 \text{ rem } (-4)$	2
Абсолютное значение	$\text{abs}(A)$	$\text{abs}(-5)$	5
Возведение в степень	$A ** B$	$5 ** 2$	25
Логическое «И»	$A \text{ and } B$	TRUE and FALSE	FALSE
Логическое «ИЛИ»	$A \text{ or } B$	TRUE or FALSE	TRUE

Логическое «НЕ»	not A	not TRUE	FALSE
Поразрядное «И»	A and B	b"0011" and b"0101"	b"0001"
Поразрядное «И-НЕ»	A nand B	b"0011" nand b"0101"	b"1110"
Поразрядное «ИЛИ»	A or B	b"0011" or b"0101"	b"0111"
Поразрядное «ИЛИ-НЕ»	A nor B	b"0011" nor b"0101"	b"1000"
Поразрядное «ИСКЛЮЧАЮЩЕЕ ИЛИ»	A xor B	b"0011" xor b"0101"	b"0110"
Поразрядное «ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ»	A xnor B	b"0011" xnor b"0101"	b"1001"
Поразрядное «НЕ»	not A	not b"1001"	b"0110"
Равенство	A = B	b"1001" = b"1001"	TRUE
Неравенство	A /= B	b"1001" /= b"1001"	FALSE
Больше	A > B	b"1001" > b"1001"	FALSE
Больше либо равно	A >= B	b"1001" >= b"1001"	TRUE
Меньше	A < B	b"1000" < b"1001"	TRUE
Меньше либо равно	A <= B	b"1000" <= b"0001"	FALSE
Логический сдвиг влево	A sll B	b"1011" sll 2	b"1100"
Логический сдвиг вправо	A srl B	b"1011" srl 2	b"0010"
Арифметический сдвиг влево	A sla B	b"1011" sla 2	b"1111"
Арифметический сдвиг вправо	A sra B	b"1011" sra 2	b"1110"
Циклический сдвиг влево	A rol B	b"1011" rol 2	b"1110"
Циклический сдвиг вправо	A ror B	b"1011" ror 2	b"1110"
Конкатенация (сцепление)	A & B	b"1011" & b"1001"	b"10111001"

Перечисленные операции определены не для всех типов данных.

6. Последовательные операторы.

В VHDL различают последовательные и параллельные операторы. Последовательные операторы аналогичны операторам обычных языков программирования. Они выполняются последовательно по мере их употребления в тексте описания объекта. Они могут использоваться внутри тел описания процессов, а также внутри тел процедур и функций. Описание последовательных операторов приводится ниже.

6.1 Оператор ожидания wait.

Оператор ожидания приводит к временной приостановке выполнения процесса, функции или процедуры. Он может быть использован в одной из трех форм:

- Ожидание условия. Приостановка до выполнения указанного условия. Примеры приведены ниже.

wait until A = B; - ожидание равенства сигналов или переменных;

wait until CLK = '1'; - ожидание «единичного уровня сигнала CLK;

wait until CLK'event and CLK = '1'; - ожидание фронта сигнала CLK;

wait until CLK'event and CLK = '0'; - ожидание среза сигнала CLK.

- Задержка. Приостановка на заданное время. Примеры приведены ниже.

wait for 25 ns; - приостановка на 25 нс;

wait for CLK_Period/2; - приостановка на половину периода сигнала CLK;

wait for (1 us – NOW); - приостановка до момента времени моделирования 1 мкс. NOW – системная переменная текущего значения времени моделирования.

Параметр CLK_Period может быть задан как константа или в списке параметров объекта Generic.

- Ожидание события. Приостановка до возникновения события в указанных сигналах или переменных. Под событием подразумевается любое изменение сигнала. Примеры приведены ниже.

wait on A; - ожидание события в сигнале A;

wait on CE, WE; - ожидание события в сигнале CE или WE.

Любые из трех описанных форм оператора **wait** могут быть использованы комбинированно в одном операторе. Примеры приведены ниже.

wait on WE until (CLK'event and clk = '1') for 10 ns; - ожидание изменения сигнала WE, затем фронта сигнала CLK и потом еще 10 нс;

wait; - бесконечный цикл ожидания.

6.2 Оператор присваивания значения переменной.

Переменная, указанная в левой части оператора принимает новое значение, задаваемое значением или выражением в правой части оператора. Примеры приведены ниже.

A := 10; - переменной A присваивается значение 10;

B := C and D; - переменной B присваивается значение выражения логической функции C and D;

E := F + G; - переменной E присваивается значение выражения суммы переменных F + G;

J := ('0', '1', others => '0'); - векторной переменной J присваивается значение, в которой старший бит равен '0', следующий – '1', и все оставшиеся – '0';

K := (others => '1'); - векторной переменной K присваивается значение, в которой все биты равны '1'.

Переменные в VHDL могут быть определены в области операторов процессов, функций и процедур.

6.2 Оператор присваивания значения сигналам.

Данный оператор служит для изменения значения сигнала, указанного в левой части оператора. Как было отмечено выше, изменение сигнала ассоциировано с конкретными моментами времени моделирования системы. Поэтому, в отличие от переменной, оно происходит не в момент выполнения оператора. В VHDL определены несколько видов задержек изменения сигнала:

- инерционная задержка;
- транспортная задержка;
- дельта-задержка.

Примеры операторов присваивания сигналов приводятся ниже.

A <= B after 5 ns;

C <= transport D after 4 ns;

Необязательное ключевое слово **transport** определяет транспортную задержку изменения сигнала. Эта задержка имитирует конечное время распространения сигнала в системе. В этом случае сигнал изменяет свое значение с задержкой, указанной после ключевого слова **after**. При этом время удержания уровня входного сигнала не имеет значения.

Если ключевое слово **transport** не указано, то необязательное ключевое слово **after** определяет инерционную задержку изменения сигнала. Смысл инерционной задержки состоит в том, что изменение сигнала будет иметь место, только если входной сигнал, указанный в правой части оператора, сохраняет свой уровень в течение указанного времени. Это свойство используется для имитации работы реальной электронной системы, обладающей паразитными емкостями на входах. Для перезаряда этих емкостей новым уровнем сигнала он должен быть удержан в новом состоянии в течение определенного времени. Иначе система не отреагирует на изменение сигнала.

Если в операторе не указано ключевое слово **after**, сигнал все равно не изменяется мгновенно. В этом случае используется т.н. дельта-задержка, которая представляет собой время одного цикла моделирования объекта. Она нужна для подтверждения причинно-следственных связей изменения сигналов. Она символизирует тот факт, что ни одна реальная система не может мгновенно отреагировать на изменение входных сигналов.

Текущее значение времени моделирования определяется системной переменной **NOW**.

В правой части оператора может быть указан другой сигнал, переменная или выражения с использованием других сигналов или переменных. Примеры приведены ниже.

```
A <= B + C;  
D <= E and F after 5 ns;
```

6.3 Условный оператор **if**.

Условный оператор позволяет обеспечить селективное выполнение операторов при выполнении или невыполнении заданных условий. Пример условного оператора приведен ниже.

```
if (Enable = '1') then Data_Out <= Data_In;  
else Data_Out <= '0';  
end if;
```

Проверяемое условие указывается после директивы **if**. В данном случае это условие (Enable = '1'). Если условие принимает значение TRUE, то выполняется оператор, указанный после директивы **then (Data_Out <= Data_In;)**. Если условие принимает значение FALSE, то указанный оператор не выполняется. Вместо него выполняется оператор, указанный после директивы **else (Data_Out <= '0';)**. Секция **else...** является необязательной. Она может отсутствовать, но если она присутствует в операторе – она может быть только одна. Для вложенности данной секции существует специальная форма, пример которой приведен ниже.

```
if (Select = 1) then Data_Out <= Data_1;  
elsif (Select = 2) then Data_Out <= Data_2;  
elsif (Select = 3) then Data_Out <= Data_3;  
elsif (Select = 4) then Data_Out <= Data_4;  
end if;
```

Заканчиваться условный оператор должен директивой **end if**.

6.4 Оператор выбора **case**.

Оператор выбора является альтернативой многократно вложенного условного оператора. Он позволяет выбрать одну из альтернатив в зависимости от конкретного значения условия. Пример оператора приведен ниже.

```
case SEL_A is  
  when 0 => Data <= Data_A;  
  when 1 =>  
    if (SEL_B = 0) then Data <= Data_B;  
    else Data <= Data_C;  
    end if;  
  when others =>  
    Data <= Data_D;  
end case;
```

Если сигнал SEL_A равен 0, то сигналу Data присваивается значение сигнала Data_A. Если сигнал SEL_A равен 1, то выполняется условный оператор **if**, в результате которого сигналу Data присваивается значение сигнала Data_B или Data_C. При всех остальных значениях сигнала SEL_A сигналу Data присваивается значение сигнала Data_D. Принципиальным условием является учет всех возможных вариантов значений сигнала или выражения, используемого в качестве селектора. Для этого может быть использована директива **others**, которая покрывает все значения выражения выбора, не перечисленные в предыдущих альтернативах.

6.5 Оператор цикла **loop**.

Оператор цикла позволяет выполнить некоторую последовательность операторов последовательно некоторое конечное или бесконечное количество раз. В VHDL

существует три формы оператора цикла. Первая форма, реализующая бесконечный цикл, представлена ниже.

```
l1: loop  
    wait until CLK'event and CLK = '1';  
    COUNT <= COUNT + 1;  
end loop l1;
```

Организация цикла начинается с директивы **loop**. В случае необходимости данная директива может иметь метку (в данном случае l1), но она не является обязательной. Далее следует т.н. «тело цикла», которое состоит из последовательности операторов, которые будут выполняться циклически. Завершается тело цикла директивой **end loop**.

Вторая форма оператора, реализующая цикл со счетчиком представлена ниже.

```
l2: for i in 0 to 15 loop  
    A(i) <= B(i+1);  
End loop l2;
```

Такая форма цикла задается директивой **for** и заданием начального и конечного значений т.н. «счетчика цикла» (в данном случае i). В данном случае на каждом очередном проходе тела цикла значение счетчика увеличивается на единицу. Выполнение цикла завершается когда значение счетчика выйдет за указанный диапазон. В приведенном примере тело цикла будет выполнено 16 раз. Переменную счетчика цикла не нужно объявлять дополнительно и ей нельзя выполнять присваивания.

Третья форма оператора, реализующая цикл с условием представлена ниже.

```
while ENABLE = '1' loop  
    DATA_OUT <= DATA_IN;  
    COUNT <= COUNT + 1;  
end loop;
```

Данная форма задается директивой **while** и заданием условия выполнения тела цикла. Перед каждым проходом тела цикла проверяется выполнение условия. Если оно принимает значение TRUE, то очередной проход цикла выполняется. Если условие принимает значение FALSE, то выполнение цикла завершается.

Циклы любой формы могут быть вложенными. Пример вложенных циклов приведен ниже.

```
l_outer: for i in 0 to 15 loop  
    l_inner: for j in 0 to 15 loop  
        A(i, j) <= B(j, i);  
    end loop l_inner  
end loop l_outer;
```

6.6 Оператор возврата return.

Оператор возврата применяется для завершения выполнения самого внутреннего цикла или самой внутренней процедуры или функции.

6.7 Оператор прерывания текущей итерации цикла next.

Данный оператор используется для прерывания выполнения текущей итерации цикла и перехода к началу следующей итерации. Пример использования оператора приведен ниже.

```
COUNT := 0;  
for i in 0 to 15 loop  
    next when A(i) = '0';  
    COUNT := COUNT +1;  
end loop;
```

В данном примере подсчитывается количество единиц в векторе A. При обнаружении нулевого значения в векторе A выполнение данной итерации цикла прекращается и начинается следующая итерация. Пример взят с сайта <https://kanyevsky.kpi.ua/>.

6.8 Оператор прерывания цикла exit.

Этот оператор предназначен для досрочного завершения выполнения цикла при выполнении заданного условия. Пример использования оператора приведен ниже.

```
COUNT := 0;
for i in 0 to 15 loop
    exit when A(i) = '1';
    COUNT := COUNT + 1;
end loop;
```

В данном примере находится номер самой левой единицы в векторе А. При обнаружении первой же единицы в векторе А выполнение цикла прекращается.

Пример взят с сайта <https://kanyevsky.kpi.ua/>.

6.9 Оператор определения процедуры **procedure**.

Процедуры в VHDL играют ту же роль, что и в классических языках программирования. Они используются для сокращения кода описания объекта в тех случаях, когда в разных местах описания неоднократно необходимо повторить одну и ту же последовательность операторов. Пример описания процедуры приведен ниже.

```
procedure sort2(variable x1,x2:inout integer) is
variable t:integer;
begin
  if x1>x2 then
    return;
  else
    t:=x1; x1:=x2; x2:=t;
  end if;
end procedure;
```

Пример взят с сайта <https://kanyevsky.kpi.ua/>.

В операторе **procedure** в скобках задается список формальных параметров. В качестве формальных параметров могут использоваться константы, переменные или сигналы. Для каждого параметра задается его тип и направление: in – входной, out – выходной, inout – двунаправленный. В теле процедуры изменять можно только значения выходных и двунаправленных формальных параметров, которые являются возвращаемыми. Т.о., процедура может возвращать сразу несколько значений. Оператор **return** выполняет здесь ту же роль, что и в циклах, т.е. немедленное завершение процедуры и выход из нее. В процедуре он является необязательным. Завершаться определение процедуры должно директивой **end procedure**. Вызов объявленной процедуры осуществляется директивой, приведенной ниже.

```
sort2(b1,b2);
```

В ней вместо формальных параметров указываются реальные.

6.10 Оператор определения функции **function**.

Функция выполняет ту же роль, что и процедура, но отличается от нее логикой оформления. Пример определения функции приведен ниже.

```
function PAR_GEN (BV : bit_vector) return bit is
variable PAR : bit := '0';
begin
  for i in BV'range loop
    PAR := PAR xor BV(i) ;
  end loop ;
  return PAR ;
end function PAR_GEN;
```

В качестве формальных параметров функции могут задаваться только константы или сигналы. Параметры могут только входными, значения которых нельзя изменять в теле функции. В отличие от процедуры функция возвращает только одно значение, тип

которого задается после слова **return** в операторе определения функции, а сам возврат осуществляется оператором **return** в теле функции. Поэтому, оператор **return** в теле функции является обязательным. Вызов функции осуществляется директивой, приведенной ниже.

```
PAR_DATA <= PAR_GEN (DATA);
```

В ней вместо формальных параметров указываются реальные.

6.11 Оператор сообщения **assert**.

Оператор сообщения предназначен для диагностики контрольных ситуаций при моделировании системы. Он выводит на консоль моделирующей программы сообщение если условие, указанное в операторе, не выполняется. Пример оператора приведен ниже.

```
Assert (CLK'event and clk='1') report "CLK rising edge is not present" severity WARNING;
```

Выводимое сообщение указывается в двойных кавычках после директивы **report**. Директива **severity** позволяет конкретизировать степень важности сообщения, которая может быть определена как: **NOTE**, **WARNING**, **ERROR** или **FALURE**.

6.12 Оператор сообщения **report**.

Если необходимо вывести сообщение о ходе работы объекта при любых условиях используется оператор **report**. Пример использования оператора приведен ниже.

```
report "Hello, world" severity NOTE;
```

Директива **severity**, как и в предыдущем случае, позволяет конкретизировать степень важности сообщения.

6.13 Пустой оператор **null**.

Этот оператор не выполняет никаких действий. Его смысл заключается в конкретизации отсутствия действий в определенном случае. Пример использования оператора приведен ниже.

```
case SEL is
begin
    when 0 => y <= 0;
    when 1 => NULL;
end case;
```

В данном примере при значении переменной SEL, равном 1, не выполняется никаких действий.

7. Параллельные операторы.

Параллельные операторы являются отличительной особенностью языков описания аппаратуры. В любой аппаратной системе может одновременно функционировать множество ее блоков (объектов) и изменяться множество сигналов. Эту особенность аппаратных систем в языках описания аппаратуры призваны отразить параллельные операторы. Несмотря на их расположение в разных частях описания объекта предполагается, что их выполнение может быть привязано к одному и тому же времени моделирования объекта. Это создает иллюзию их одновременного (параллельного во времени) выполнения.

7.1 Оператор процесса **process**.

Под процессом понимается описание поведения некоторой независимой части объекта с помощью последовательных операторов. Независимо от количества последовательных операторов в теле процесса во время его выполнения время

моделирования останавливается. Для описания процесса применяется оператор **process**. Пример декларирования процесса приведен ниже.

```
GEN1: process
constant CLK_Period: time:= 100 ns;
begin
    if now = 0 ns then CLK <= '0';
    end if;
    wait for CLK_Period/2;
    CLK <= '1';
    wait for CLK_Period/2;
    CLK <= '0';
end process GEN1;
```

Метка в операторе процесса не является обязательной (в данном случае GEN1) не является обязательной. Внутри тела процесса можно включать декларацию констант, типов, переменных, тел процедур или функций. Сигналы нельзя декларировать внутри тела процесса. Если нужна декларация указанных элементов – ее нужно делать до директивы **begin**. После этой директивы начинается алгоритмическая часть тела процесса, где описывается его функционирование. Внутри нее используются последовательные операторы. Заканчивается описание тела процесса директивой **end process**. В данном примере при нулевом времени моделирования сигналу **CLK** присваивается нулевое значение. Далее с периодичностью **CLK_Period/2** (5 нс) инвертируется значение сигнала **CLK**. Таким образом, формируется периодический сигнал с частотой 10 МГц.

Любой процесс может находиться либо в активном, либо в пассивном состоянии. В пассивном состоянии процесс находится в ожидании выполнения условия, указанного в операторе **wait** или изменения какого-либо сигнала, указанного в списке чувствительности процесса. Список чувствительности если он есть указывается в скобках в операторе **process**. Пример приведен ниже.

```
process (CLK)
begin
    if (CLK'event and CLK = '1') then D_OUT <= D_IN;
end process;
```

В данном примере по положительному перепаду сигнала **CLK** сигналу **D_OUT** присваивается значение сигнала **D_IN**. Таким образом, описана работа D-триггера. В представленном примере процесс находится в пассивном состоянии, пока не произойдет изменение сигнала **CLK**, который указан в списке чувствительности. При возникновении любого изменения сигнала **CLK** процесс активизируется и выполняется его алгоритмическая часть.

Если список чувствительности в операторе **process** отсутствует, то процесс активизируется при изменении любого сигнала, используемого в его алгоритмической части.

7.2 Оператор условного назначения сигнала.

Оператор предназначен для назначения значения сигналу в зависимости от выполнения указанных в нем условий. Пример оператора приведен ниже.

```
DATA <= INPUT_1 when ((ADDRESS = "00") or (CS_1='0')) else
    INPUT_2 when ((ADDRESS = "01") or (CS_2='0')) else
    INPUT_3 when ((ADDRESS = "10") or (CS_3='0')) else
    INPUT_4;
```

Проверяемое условие указывается после директивы **when**. В приведенном примере сигналу **DATA** присваивается значение сигнала **INPUT_1** если сигнал **ADDRESS** равен "00", сигнала **INPUT_2** если сигнал **ADDRESS** равен "01", сигнала **INPUT_3** если сигнал **ADDRESS** равен "10", сигнала **INPUT_4** в других случаях. Аналогичный алгоритм присвоения можно реализовать с помощью операторов **if**, который является аналогом

описываемого оператора. Однако оператор **if** является последовательными и для его использования необходимо объявить отдельный процесс, внутри которого реализовать указанный алгоритм присвоения. С помощью условного оператора присвоения этот алгоритм можно реализовать без объявления процесса. Условия в разных секциях **when** могут различными и содержать разные сигналы и переменные.

7.3 Оператор выборочного назначения сигнала.

Если во всех условиях назначения сигнала в предыдущем примере используется одно и то же выражение, то вместо него можно использовать оператор выборочного назначения сигнала. Пример приведен ниже.

```
with ADDRESS select
DATA <= INPUT_1 when "00",
      INPUT_2 when "01",
      INPUT_3 when "10",
      INPUT_4 when others;
```

Этот оператор является аналогом последовательного оператора **case**, но может быть использован без объявления процесса.

7.4 Оператор параллельного вызова процедуры.

Это обычный оператор вызова процедуры, синтаксис которого описан в 6.9, который применяется вне описания процесса. При этом все формальные параметры процедуры должны быть константами или сигналами. Используемая процедура должна быть объявлена в этой же части описания архитектуры объекта. Пример приведен ниже.

```
entity SYS_1 is
port(D_IN: in std_logic_vector(7 downto 0);
      D_OUT: out std_logic_vector(1 downto 0));
end SYS_1;
architecture AR of SYS_1 is
procedure OR_4(signal IN_1, IN_2, IN_3, IN_4:in std_logic;
               signal Q:out std_logic) is
begin
Q <= IN_1 or IN_2 or IN_3 or IN_4;
end OR_4;
begin
OR_4(D_IN(0), D_IN(1), D_IN(2), D_IN(3), D_OUT(0));
OR_4(D_IN(4), D_IN(5), D_IN(6), D_IN(7), D_OUT(1));
end AR;
```

7.5 Оператор параллельного сообщения assert.

Это оператор, аналогичный описанному в 6.11, который используется вне тела описания процесса.

7.6 Оператор генерации generate.

Оператор генерации имеет смысл использовать в тех случаях, когда в системе используется относительно большое количество одинаковых блоков. В таком случае удобно описать многократно используемый блок как компонент с помощью оператора **component**, как описано в главе 4. Если количество используемых экземпляров данного компонента относительно небольшое, то их можно конкретизировать обычным способом. Однако, если необходимо объявить большое число экземпляров компонента, это существенно может увеличить объем текста описания, что, помимо прочего, ухудшает его «читабельность». Оператор **generate** позволяет компактно в цикле конкретизировать необходимое количество экземпляров компонента. Пример использования оператора в синтаксисе оператора **for loop** приведен ниже.

```
entity SYS_2 is
generic (Data_Width: integer:= 16);
port(A_IN: in std_logic_vector(Data_Width - 1 downto 0);
```

```

        B_IN: in std_logic_vector(Data_Width - 1 downto 0);
        D_OUT: out std_logic_vector(Data_Width - 1 downto 0));
end SYS_2;
architecture AR of SYS_2 is
component S1 is
port(IN_1, IN_2:in std_logic;
      Q:out std_logic);
end component;
begin
G1: for I in 0 to Data_Width - 1 generate
      G2:S1 port map(IN_1 => A_IN(I), IN_2 => B_IN(I), Q => D_OUT(I));
end generate;
end AR;

```

В приведенном примере количество необходимых экземпляров компонента задается параметром `Data_Width` в секции **generic** описания объекта. Как видно в примере, оператор **generate** в цикле конкретизирует с соответствующим распределением входов и выходов необходимое количество (в данном случае 16) компонента S1. В случае необходимости изменения требуемого количества экземпляров компонента, это можно легко сделать изменением значения параметра `Data_Width` в секции **generic**, без необходимости коррекции остального текста описания объекта.

Другой способ использования оператора генерации в синтаксисе оператора **if** представлен в приведенном ниже примере.

```

RES0: if PULLUP_EN = 1 generate
      RES1:for i in DATA_BUS'range generate
            U_RES: PULLUP(DATA => DATA_BUS(i));
      end generate;
end generate;

```

Пример взят с сайта <https://kanyevsky.kpi.ua/>.

В приведенном примере производится проверка значения переменной `PULLUP_EN`. Если она равна единице, то производится конкретизация экземпляров компонента `PULLUP` в количестве, определяемом атрибутом **range** (диапазон) сигнала `DATA_BUS`. Если значение `PULLUP_EN` не равно единице, то генерация экземпляров компонента не производится.

7.7 Оператор определения блока **block**.

Блоки используются для разбиения описания объекта на иерархические структурные части. Такое разбиение можно использовать для ограничения области действия переменных и процедур, описанных внутри блока. В общем случае под блоком понимается часть кода описания объекта, состоящий из описательной и алгоритмической частей. Описательная часть может содержать декларации констант, сигналов, процедур и функций. Алгоритмическая часть блока содержит параллельные операторы, описывающие функционирование блока. Блоки могут быть вложенными друг в друга. Оператор определения блока может содержать т.н. «охранное выражение», имеющее значение типа `BOOL`, которое используется для модификации алгоритма работы блока. В операторе определения блока обязательна метка. Описание блока должно завершаться директивой **end block** с указанием метки блока. Пример определения блока приведен ниже.

```

RD: block ((SEL = '0') and (READ = '0')) is
begin
  DATA <= REG_0 when (guard and (ADDR = '0'))
else
  REG_1 when (guard and (ADDR = '1'))
else
  (others 'Z');
end block RD;

```


Охранное выражение, заявленное в операторе **block** ((SEL = '0') and (READ = '0')) в алгоритмической части описания блока обозначается как **guard**. В данном примере оно используется в операторе условного назначения сигнала. Другой способ использования охранного выражения иллюстрируется в следующем примере.

```
SUM: block (ENABLE = '1') is
Begin
SUM <= guarded (a(0) xor b(0));
CARRY <= guarded (a(0) and b(0));
end block SUM;
```

В данном примере используются т.н. «охраняемые» назначения сигналов, определяемые директивой **guarded**. Если охранное выражения равно TRUE, то охраняемые назначения сигналов выполняются, а если оно равно FALSE, то нет. В данном примере это используется для включения и выключения полусумматора.

8. Пакеты и библиотеки.

При проектировании больших комплексных объектов приходится определять большое количество типов данных, констант, процедур, функций и сигналов, которые используются в разных частях описания объекта. Для исключения необходимости многократно повторять описания перечисленных ресурсов принято объединять их в «пакеты», которые затем включаются в те части описания объекта, где планируется использование их содержимого. Пакет состоит из декларативной части и тела пакета. В декларативной части определяются типы данных, констант, переменных, функции и процедуры. Если процедуры и функции не определяются, то тело пакета может отсутствовать. Пример определения декларативной части пакета приведен ниже.

```
package SYSTEM_Definitions is
type permit is (enable, disable);
type permit_vector is array (natural range <>) of permit;
type states is (passive, active);
constant CLK_Period: integer:=1;
function stdl_to_permit (input:in std_logic) return permit;
function permit_to_stdl (input:in permit) return std_logic;
```

```
end package SYSTEM_Definitions;
```

Начинается декларативная часть пакета директивой **package**, в которой указывается имя пакета (SYSTEM_Definitions). Далее следуют декларации компонентов пакета. Завершается декларативная часть пакета директивой **end package**.

Если в пакете присутствуют декларации функций и процедур, то их тела определяются в т.н. теле пакета, которое начинается директивой **package body**. Заканчивается тело пакета директивой **end**. Пример определения тела пакета приведен ниже.

```
package body SYSTEM_Definitions is
function stdl_to_permit (input:in std_logic) return permit is
variable output: permit;
begin
if input='1' then output:=enable;
else output:=disable;
end if;
return output;
end function stdl_to_permit;

function permit_to_stdl (input:in permit) return std_logic is
variable output: std_logic;
begin
```

```

if input=enable then output:='1';
else output:='0';
end if;
return output;
end function permit_to_std1;

```

end SYSTEM_Definitions;

Пакеты с определенными ресурсами объекта объединяются в библиотеки ресурсов, которые представляют собой отдельные файлы. Для использования ресурсов библиотек необходимо указать ссылку на нее в начале описания объекта. Ссылка на библиотеку задается директивой **library**. Пример ссылки на библиотеку приведен ниже.

```

library SYSTEM_LIB;
use SYSTEM_LIB. SYSTEM_Definitions.all;

```

Директива **use** предписывает использовать в описании объекта все ресурсы пакета **SYSTEM_Definitions**, который содержится в библиотеке **SYSTEM_LIB**. Если нужно использовать не все, а какой-то конкретный ресурс, описанный в библиотеке, то вместо ключевого слова **all** нужно указать его имя. Пример приведен ниже.

```

library SYSTEM_LIB;
use SYSTEM_LIB. SYSTEM_Definitions.CLK_Period;
use SYSTEM_LIB. SYSTEM_Definitions. stdl_to_permit;
use SYSTEM_LIB. SYSTEM_Definitions. permit_to_std1;

```

Файлы описания объекта, создаваемые пользователем, тоже размещаются моделирующей программой в т.н. рабочей библиотеке, которая имеет стандартное имя **work**. Перед началом моделирования дополнительно подключаются стандартные или созданные пользователем библиотеки, ссылки на которые присутствуют в файлах описания объекта.

Многие часто используемые типы данных и операции над элементами таких типов определены в конкретных библиотеках. Поэтому, для использования таких типов и операций над ними необходимо вначале описания объекта указать ссылку на соответствующие библиотеки. В частности, часто употребляемые типы данных **std_logic** и **std_logic_vector** определены в пакете **std_logic_1164** библиотеки **ieee**. Поэтому, если в описании объекта планируется использовать указанные типы данных и функции, в начале описания необходимо поместить ссылку на данный пакет как показано ниже.

```

library ieee;
use ieee.std_logic_1164.all;

```

Для выполнения различных математических и логических операций над многобитными числами без знака и со знаком в пакете **numeric_std** библиотеки **ieee** предусмотрены специальные типы данных **unsigned** и **signed** соответственно, представляющие собой подтипы (subtype) **std_logic_vector**.

Математические операции сложения, вычитания, умножения, сравнения над данными типа **std_logic_vector** как над числами без знака и со знаком определены в пакетах **std_logic_unsigned** и **std_logic_signed** библиотеки **ieee** соответственно. Функция преобразования целого числа в массив типа **std_logic_vector**, которая называется **conv_std_logic_vector(arg: integer, size: integer)**, определена в пакете **std_logic_arith** библиотеки **ieee**. Функция, реализующая обратное преобразование массива типа **std_logic_vector** в целое число без знака и со знаком, которая называется **conv_integer(arg: std_logic_vector)**, определена в пакетах **std_logic_unsigned** и **std_logic_signed** библиотеки **ieee** соответственно.

9. Конфигурация.

Как было показано в главе 4, один и тот же объект можно описать разными способами. Это означает, что для одного объекта можно создать несколько разных архитектур. Пример приведен ниже.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

Entity andnot is
Port (a:in std_logic;
       b:in std_logic;
       q:out std_logic);
End andnot;

Architecture behav_andnot of andnot is
Begin
  q <= '0' when ((a = '1') and (b='1')) else '1';
End Architecture behav_andnot;

Architecture behav1_andnot of andnot is
Begin
  q <= not (a and b);
End Architecture behav1_andnot;

```

В приведенном примере описан объект, реализующий логическую функцию «И-НЕ». Предположим, что мы хотим реализовать на таких логических элементах RS-триггер, описанный в главе 4.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;
Entity rs_trg is
Port (r:in std_logic;
       s:in std_logic;
       q:out std_logic;
       qn:out std_logic);
End rs_trg;
Architecture behav_rs_trg of rs_trg is
Component andnot is
Port (a:in std_logic;
       b:in std_logic;
       q:out std_logic);
End Component andnot;
signal q_int: std_logic;
signal qn_int: std_logic;
Begin
  nand1: andnot
  Port Map (a=>s,
            b=>qn_int,
            q=>q_int);
  nand2: andnot
  Port Map (a=>r,
            b=>q_int,
            q=>qn_int);
  q <= q_int;
  qn <= qn_int;
End Architecture behav_rs_trg;

```

Как видно в приведенном описании объявляются два экземпляра объекта andnot. Для каждого из них можно выбрать одну из описанных архитектур объекта. Для этого используется директива конфигурации. Пример приведен ниже.

```

Configuration c_rs of rs_trg is
for behav_rs_trg
    for nand1: andnot
        use entity work.andnot(behav_andnot);
    end for;
    for nand2: andnot
        use entity work.andnot(behav1_andnot);
    end for;
end for;
End c_rs;

```

В директиве конфигурации объявляется ее имя (c_rs) и указывается, для какого объекта она задается. Далее в первой директиве **for** определяется, для какой архитектуры объекта она определяется. Вторая и третья директивы **for** определяют какую их архитектур объекта andnot использовать для каждого из его экземпляров. Как видно, выбранные архитектуры расположены в рабочей библиотеке work, в которой помещаются все описания объектов, созданных пользователем. Заканчивается объявление конфигурации директивой **end**.

10. Файлы.

Файлы используются в моделях систем для хранения данных инициализации объектов (например, начальное содержимое памяти), форм тестирующих входных сигналов, контрольных наборов выходных сигналов при верификации работы моделируемых систем.

Работа с файлами иллюстрируется на примере, представленном ниже.

```

SYM_proc: Process
variable SYM_DATA: std_logic_vector(3 downto 0);
variable l: line;
variable lr: line;
variable good_val, good_number: boolean;
variable r : integer;
variable vector_time: time;
variable space: character;
file vector_file: text;
file result_file: text;
Begin
FILE_OPEN(vector_file, "values.txt", READ_MODE);
FILE_OPEN(result_file, "res.txt", WRITE_MODE);
while not endfile(vector_file) loop
    readline(vector_file, l);
    read(l, r, good => good_number);
    next when not good_number;
    vector_time := r * 1 ns;
    if (now < vector_time) then
        wait for vector_time - now;
    end if;
    read(l, space);
    read(l, SYM_DATA, good_val);
    assert good_val REPORT "bad value";
    DATA_IN <= SYM_DATA;
    wait for 1 ns;
    write(lr, now);
    write(lr, ' ');
    write(lr, DATA_OUT);
end loop;
end SYM_proc;

```

```

        writeline(result_file, lr);
    end loop;
    FILE_CLOSE(vector_file);
    FILE_CLOSE(result_file);

```

end process;

Для работы с файлами используются специальные типы данных – **file** (файл) и **line** (строка текстового файла). Для работы с файлом необходимо сначала объявить логический файл с помощью директивы, приведенной ниже.

file vector_file: text;

file result_file: text;

В данном примере объявленный логический файл имеет имя **vector_file** для файла исходных данных и **result_file** для файла результатов. Кроме этого объявляются переменные **l** и **lr** типа **line**.

variable l: line;

variable lr: line;

Далее необходимо открыть объявленный файл и связать его с именем реального физического файла. Это делается с помощью процедуры **FILE_OPEN**.

FILE_OPEN(vector_file, "values.txt", READ_MODE);

FILE_OPEN(result_file, "res.txt", WRITE_MODE);

В данной процедуре первый параметр это имя логического файла, второй параметр – имя физического файла, третий параметр – режим открытия файла:

- **READ_MODE** – режим чтения;

- **WRITE_MODE** – режим записи.

Условием цикла **loop** является достижение конца файла **vector_file**, которое определяется с помощью функции **endfile(vector_file)**.

С помощью процедуры **readline(vector_file, l)** считывается очередная строка из файла **vector_file** в переменную **l**.

Далее с помощью процедуры **read** считывается сначала значение времени, соответствующее данному значению входного сигнала, затем пробел, а затем само значение сигнала. Третий параметр процедуры **read**, если он присутствует, возвращает флаг успешного чтения значения, заданного параметра.

С помощью процедуры **write** в строку **lr** записываются значение времени моделирования, пробел и значение выходного сигнала. Затем процедура **writeline(result_file, lr)** сформированная строка записывается в файл результатов.

После завершения использования работы с файлом его необходимо закрыть. Это делается с помощью процедуры **FILE_CLOSE**.

Процедуры, функции и типы для работы с файлами определены в пакете **TEXTIO** библиотеки **STD** и в пакете **STD_LOGIC_TEXTIO** библиотеки **ieee**.

Литература:

1. П.Н. Бибило, Н.А. Авдеев. Моделирование и верификация цифровых систем на языке VHDL. «Ленанд». 2017. 344 с. с ил. ISBN - 978-5-9710-3937-2.
2. Е. А. Суворова, Ю. Е. Шейнин. Проектирование цифровых систем на VHDL. «БХВ-С.Петербург». 2003., 576 стр. с.ил. ISBN- 5-94157-189-5.
3. П.Н. Бибило. Основы языка VHDL. «Ленанд». 2020. 328 с. с ил. ISBN - 978-5-9710-7135-8.
4. П.Н. Бибило. Синтез логических схем с использованием языка VHDL. «Солон-Пресс». 2002. 384 с. с ил. ISBN - 978-5-93455-152-3.
5. А.К. Поляков. Языки VHDL и VERILOG в проектировании цифровой аппаратуры. «Солон- Пресс». 2003. 313 с. с ил. ISBN - 978-5-98003-016-36.